

RELATÓRIO TÉCNICO: AEP 4º SEMESTRE (2026.2)

Projeto: FoodShare – Central de Gerenciamento de Doações

Cursos: Engenharia de Software / Análise e Desenvolvimento de Sistemas

Integrantes:

- Renan Madeira Evaristo - RA: 25011033-2
 - Samuel Felipe Narciso - RA: 25247839-2
 - José Carlos de Mello Junior - RA: 25357053-2
-

1. CONCEPÇÃO, ALINHAMENTO ODS E DESCOBERTA

- **ODS Vinculada: ODS 2 – Fome Zero e Agricultura Sustentável** (Meta 2.1 e Meta 12.3 de redução do desperdício de alimentos).
- **Problema Identificado (Dor):** Grandes volumes de alimentos perdem seu valor comercial antes da data de validade (ex.: produtos próximos ao vencimento, excedentes de produção ou hortifrúti fora do padrão estético), acabando descartados em aterros. Paralelamente, ONGs e cozinhas comunitárias enfrentam escassez de suprimentos por falta de um canal centralizado e ágil para identificar a oferta desses donativos em tempo hábil.
- **Proposta da Solução:** O **FoodShare** é uma plataforma que conecta estabelecimentos comerciais doadores (supermercados, restaurantes, padarias) a entidades receptoras cadastradas, automatizando o controle de lotes, tipos de perecibilidade e status da logística de coleta.

Lista de Requisitos (Escopo do Projeto)

1. O sistema deve permitir o cadastro de usuários doadores, contendo CNPJ, Razão Social, Nome Fantasia, Endereço completo, Telefone comercial e E-mail de contato.
2. O sistema deve permitir o cadastro de entidades receptoras (ONGs), contendo CNPJ, Nome da Instituição, Nome do Responsável, Endereço de entrega, Telefone e Capacidade diária de recebimento.
3. O sistema deve permitir o cadastro de itens para doação, contendo Descrição do alimento, Categoria (Perecível ou Não Perecível), Quantidade, Unidade de medida (kg, caixas, unidades) e Data limite de consumo/validade.
4. O sistema deve permitir a consulta e filtragem de doações disponíveis com base na categoria do alimento (Perecível / Não Perecível) e prazo de validade.
5. O sistema deve permitir a atualização e acompanhamento do ciclo de vida da doação através de status controlados: DISPONIVEL, EM_TRANSITO, FINALIZADA e CANCELADA.
6. O sistema deve emitir alertas automáticos de notificação polimórfica para os envolvidos sempre que houver alteração de status na coleta.

2. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

- **Linguagem (Java):** Escolhida pela tipagem estática e forte adesão aos conceitos da Programação Orientada a Objetos (POO). Facilita o uso rigoroso de herança (reaproveitamento da classe base Usuario), encapsulamento, interfaces e sobrescrita de métodos (@Override), cumprindo as métricas acadêmicas exigidas.
- **Banco de Dados (MySQL):** Escolhido por ser um banco relacional robusto, gratuito e muito utilizado. Ele garante a segurança dos dados ao relacionar as tabelas (impedindo, por exemplo, que existam doações cadastradas sem um doador vinculado). A conexão com o Java será feita utilizando **JDBC**, permitindo executar os comandos de cadastro, busca e atualização com segurança e estabilidade.
- **Padrão Arquitetural (MVC):**
 - **Model:** Classes de domínio (Usuario, Doador, Instituicao, Doacao) e classes DAO para persistência no banco.
 - **View:** Camada de console/terminal para entrada e saída de dados, simples e sem dependência de UI gráfica complexa.
 - **Controller:** Centraliza as regras de negócio, validações de validade/datas e a mediação entre a interface e os DAOs.
- **Compatibilidade e Execução (Sistemas Operacionais):** Construção orientada ao Java SE com suporte multiplataforma (Linux, Windows, macOS). O repositório conterá instruções de compilação via terminal (javac/java ou Maven) e o script SQL (schema.sql) para inicialização de banco em qualquer ambiente.

3. CRONOGRAMA DE EXECUÇÃO

Data / Período	Atividade	Responsável
Bimestre 1 - Sem. 1 e 2	Levantamento de escopo, especificação de requisitos e alinhamento ODS	Todos os Integrantes
Bimestre 1 - Sem. 3	Criação do repositório GitHub, estrutura de diretórios e escrita do README.md	Renan
Bimestre 1 - Sem. 4	Modelagem UML (Diagrama de Classes com 1:N, Herança e Polimorfismo)	Samuel
Bimestre 1 - Sem. 5	Modelagem do Banco de Dados Relacional (DER) e Script DDL inicial	José

Bimestre 1 - Sem. 6	Consolidação do Relatório Técnico da 1ª Entrega em PDF	Todos os integrantes
Bimestre 2 - Sem. 1 e 2	Implementação das classes Model, interfaces e hierarquias de POO	Renan
Bimestre 2 - Sem. 3 e 4	Configuração do JDBC e desenvolvimento do CRUD completo da entidade Doação	Samuel
Bimestre 2 - Sem. 5	Construção da camada Controller e View (Console)	José
Bimestre 2 - Sem. 6	Testes de integração, gravação da demonstração e commit final do código	Todos os integrantes

4. AMBIENTE DE CONTROLE DE VERSÃO (GITHUB)

- **Link Público do Repositório:**
<https://github.com/RenanMEvaristo/FoodShare-AEP.git>
- **Estrutura de Pastas Implementada:**

```

foodshare-aep/
├── /docs/
│   ├── diagrama_classes.png
│   ├── diagrama_der.png
│   └── relatorio_aep_etapa1.pdf
├── /database/
│   ├── schema.sql
│   └── seeds.sql
├── /src/
│   └── (estrutura de pacotes Java reservada: model, view, controller, dao)
├── .gitignore
└── README.md

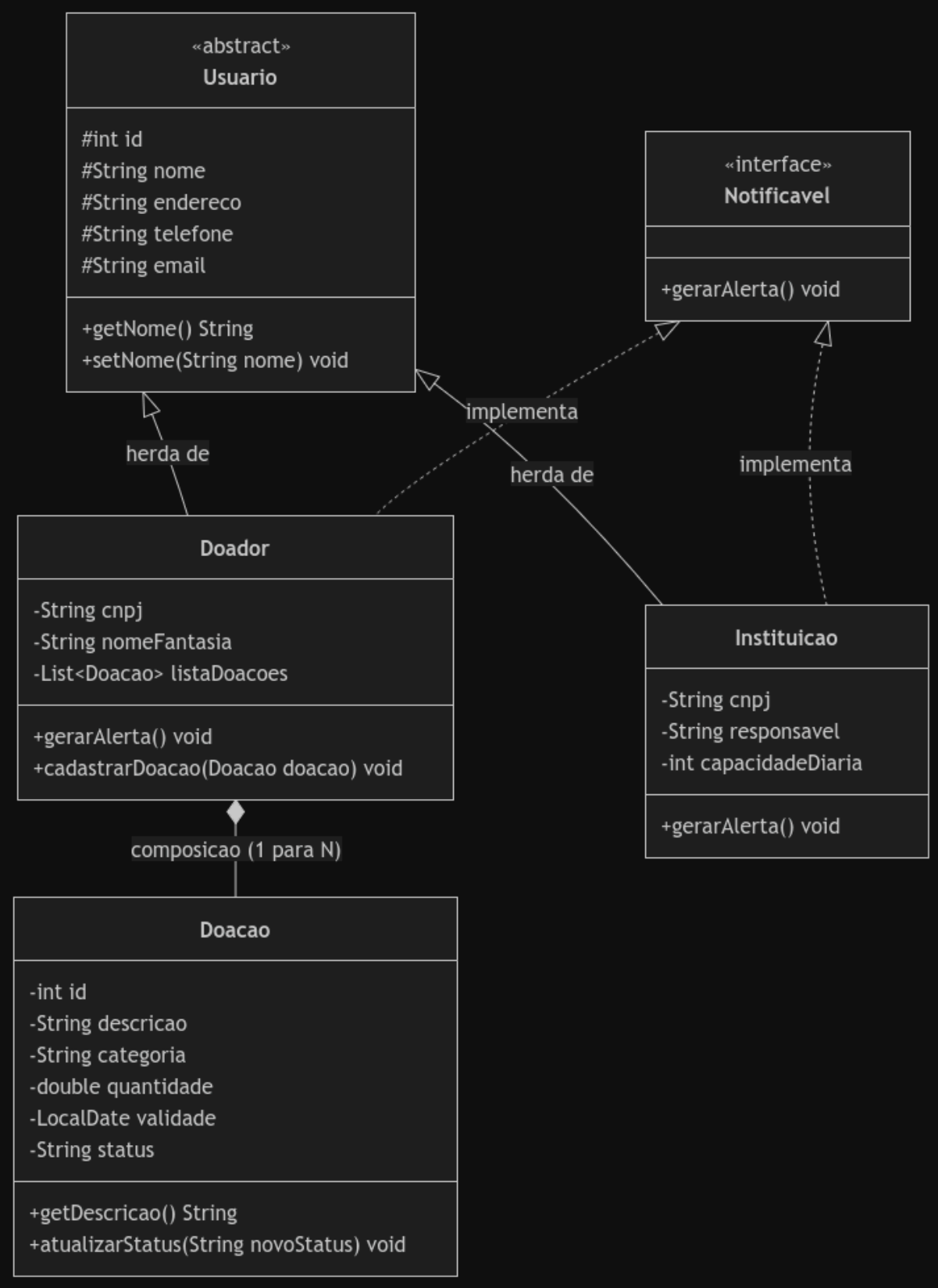
```

5. PROJETO ARQUITETURAL DIAGRAMA DE CLASSE

A. Requisitos do Diagrama de Classes

Para atender integralmente aos pilares de POO cobrados:

1. **Classe Abstrata:** Usuario (atributos: id, nome, endereco, telefone, email; método abstrato `exibirPerfil()`).
2. **Herança:**
 - Doador extends Usuario (atributos extras: `cnpj`, `nomeFantasia`, `listaDoacoes`).
 - Instituicao extends Usuario (atributos extras: `cnpj`, `responsavel`, `capacidadeDiaria`).
3. **Composição (Multiplicidade 1:N):**
 - A classe Doador possui uma composição com Doacao: `List<Doacao> listaDoacoes`. Se o doador for removido do sistema, suas doações associadas são removidas.
4. **Interface e Polimorfismo:**
 - Interface `Notificavel` com o método `void enviarAlerta(String mensagem)`.
 - Tanto Doador quanto Instituicao implementam `Notificavel`, sobrescrevendo o método com `@Override`:
 - `Doador.enviarAlerta()`: imprime confirmação de coleta agendada.
 - `Instituicao.enviarAlerta()`: imprime aviso de nova oferta de alimentos disponível.



B. Diagrama Entidade-Relacionamento (DER / Banco de Dados)

Estrutura das tabelas no MySQL:

- **usuarios:** id_usuario (PK, AUTO_INCREMENT), nome, endereco, telefone, email, tipo_usuario (ENUM: 'DOADOR', 'INSTITUICAO').
- **doadores:** id_doador (PK), id_usuario (FK -> usuarios.id_usuario), cnpj, nome_fantasia.
- **instituicoes:** id_instituicao (PK), id_usuario (FK -> usuarios.id_usuario), cnpj, responsavel, capacidade_diaria.
- **doacoes:** id_doacao (PK, AUTO_INCREMENT), id_doador (FK -> doadores.id_doador), descricao, categoria (Perecível/Não Perecível), quantidade, validade, status ('DISPONIVEL', 'EM_TRANSITO', 'FINALIZADA', 'CANCELADA').

